Insights and Enhancements in Perceptron Diabetes Prediction

Shashikant Chaudhary
University of Adelaide

Abstract

This paper investigates the application of the brain-inspired algorithm, called Perceptron. This algorithm has been used to predict the likelihood of Diabetes using the Pima Indians Diabetes Database. The trained model is evaluated on several parameters such as accuracy, precision, recall, and F1 score. Five-fold cross-validation is also used to check the robustness of the model. We show the effectiveness of this algorithm by showing its effectiveness of correctly predicting diabetic risk from the dataset. The standard perceptron model has achieved an accuracy of 67%, and then by introducing sigmoid and loss functions, we have increased the accuracy to 77%. Later we addressed the imbalanced class and then changed the algorithm by introducing class weights and precision, recall values increased significantly to 66%. Additionally, we have compared the perceptron algorithm with other modern algorithms to find the relative strengths and weaknesses of the algorithm. This work underlines the potential of the Perceptron as a baseline model for classification tasks and provides insights into performances and evaluation techniques.

1. Introduction

Diabetes is a global problem and affects a significant number of people in the world leading to high healthcare costs and long-term health complications affecting other parts of the body such as vision problems, kidney failure, heart disease, nerve damage, etc. Early detection of such disease in an individual is crucial and it is important to know what factors are significant contributors leading to diabetes and what could be possible physiological factors contribute to diabetes. Machine learning techniques are very helpful in finding patterns in the data and predicting if someone can have diabetes [1].

To solve such a problem we can use the simple brain-inspired algorithm introduced by American psychologist Frank Rosenblatt [2]. The perceptron model he introduced is a linear classifier, meaning it can be used to separate the data using a hyperplane. The algorithm is inspired by the functioning of biological neurons, and it is the foundation of more advanced Neural Network architectures which is being used in the present day. The perceptron model is not useful in cases where data is not linearly separable, meaning you cannot separate the data by drawing a hyperplane, but this is an effective baseline model for many classification tasks.

The main contributions of the paper are as follows:

- Description of the Perceptron algorithm and then its implementation on the Pima Indians Diabetes Database [3].
- Base model changes in pursuance of an investigation into potential improvements.
- Comparing with other Machine Learning Algorithms.
- Models' convergence property and performance metrics analysis.

2. Method

The Perceptron Algorithm was first introduced by the American Psychologist Frank Rosenblatt in 1958 which laid the foundation for Artificial Neural Networks. This algorithm was inspired by the working of a biological neuron and tried to simulate, how a brain processes information [2]. This algorithm was introduced for doing binary classification tasks.

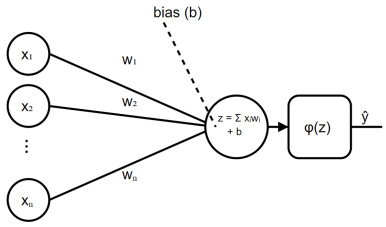


Figure 1. Perceptron Model. The figure illustrates the basic structure of a perceptron, including inputs, weights, and activation function.

2.1. Pseudocode of Perceptron Algorithm

Below is the pseudocode for the Perceptron algorithm:

Algorithm 1 Perceptron Algorithm

Input: Dataset $D = \{(x_i, y_i)\}$ where $y_i \in \{+1, 0\}$, Learning rate η, Maximum iterations max_iter
Output: Weight vector w, Bias b
Initialize $w = [0, 0, \ldots, 0]$, $b = 0$
for $iteration = 1$ to max_iter **do**
    for each (x_i, y_i) in D **do**
        Compute $z = w \cdot x_i + b$
        Compute $\hat{y}_i = 1$ if $z \geq 0$, else 0
        Compute $error = y_i - \hat{y}_i$
        if $error \neq 0$ **then**
            Update $w \leftarrow w + \eta * error * x_i$
            Update $b \leftarrow b + \eta * error$
        end if
    end for
end for
return w, b

2.2. Mathematical Explanation

Let $\mathbf{x} = (x_1, x_2, \ldots, x_n)$ be the input vector of the dataset where x_i is an individual feature. The perceptron model computes a weighted sum of these input vectors as:

$$z = \mathbf{w} \cdot \mathbf{x} + b = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

where n is the number of input features. Here:

- $w_1, w_2, \ldots, w_n$ is the weight vector associated with the input features acting as the slope in the higher dimension plane.
- b is the bias term, which shifts the decision boundary passing through the center of the coordinate plane.
- z is the resulting output from the combination of the input and weights.

2.2.1 Activation Function

As mentioned by Haykin, perceptron uses the step function on the value of z to make the prediction [4]. It is a piecewise discontinuous function that returns values of +1 or -1 for the input fed into it:

$$\hat{y} = \phi(z) = \begin{cases} +1 & \text{if } z \geq 0 \\ -1 & \text{if } z < 0\,. \end{cases}$$

2.2.2 Decision Boundary

The perceptron tries to find a hyperplane that acts as a decision boundary in the feature space. The equation of the decision boundary is defined as:

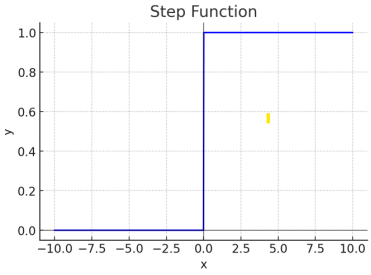


Figure 2. Step Activation Function

$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

Points on one side decision boundary are classified as $+1$ class and on the other side, it is classified as 0 class.

2.2.3 Learning Rule and Weight Update

The core idea of perceptron is to adjust its weights based on misclassification. The motive is to minimize errors by iteratively updating its weights. In simpler terms, if the model predicts wrong, the weights are adjusted in such a way that the model is more likely to predict correctly the next time.

The update rule for a given input vector and its label (x_i, y_i) is given by:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta(y_i - \hat{y}_i)\mathbf{x}_i$$

$$b \leftarrow b + \eta(y_i - \hat{y}_i)$$

Here:

- η is a learning rate, it is a constant value that determines, how large or small the updates to the weights are going to be.
- y_i is the actual label of the training example
- $\hat{y}_i$ is the predicted label.

As you can observe when the prediction is correct i.e. $y_i = \hat{y}_i$ then, the term $(y_i - \hat{y}_i)$ becomes 0, and no change is made in weights.

2.2.4 Training or Updating Weights

The perceptron algorithm works by iterating multiple times on the dataset, and updating its weights whenever any misclassification happens [4]. The steps followed in the training process are as follows.

- Compute the prediction $\hat{y}$ using the current weights.
- Update the weights if the prediction is wrong.
- Repeat the process until no change in the weight or specific number of times.

2.2.5 Convergence

Rosenblatt's original perceptron convergence theorem states that if the training data is linearly separable then the perceptron algorithm will converge to a finite set of steps i.e. it will be able to classify all points correctly. In case data is not linearly separable then the algorithm cannot converge [5].

2.3. Dataset

We have used the Pima Indians Diabetes dataset [3]. The dataset comes from research aimed at predicting diabetes in a population of women of Pima Indian heritage living near Phoenix, Arizona, in the United States. It contains 8 features and 1 output variable (0: No diabetes, 1: Diabetes). The features are summarized in the table below:

Attribute	Description	Range of Values
Pregnancies	Number of times pregnant	0 to 17
Glucose	Plasma glucose concentration (mg/dL)	0 to 199
BloodPressure	Diastolic blood pressure (mm Hg)	0 to 122
SkinThickness	Triceps skin fold thickness (mm)	0 to 99
Insulin	2-Hour serum insulin (mu U/ml)	0 to 846
BMI	Body mass index (kg/m^2)	0.0 to 67.1
DiabetesPedigreeFunction	Diabetes likelihood	0.078 to 2.42
Age	Age (years)	21 to 81
Outcome	Class variable (0 or 1)	0 or 1

Table 1. Features of the Pima Indians Diabetes Dataset.

In our paper, we have used the scaled version of the dataset [6]. This dataset contained missing values, which were imputed using the KNN Algorithm [7].

2.4. Proposed Method for Improvements

To improve the baseline model, we have gone through several experiments to improve its performance in different metrics. Later in the sections, we implemented the sigmoid function and binary cross entropy loss so that weight updates can become smooth [4]. Additionally, we introduced class weights to handle imbalanced datasets.

Parameter	Data Split	Learning Rate	Epochs	Cross-Validation
Value	80% for Training, 20% for Test	0.01	100	5-fold

Table 2. Training Parameters

To evaluate the performance of our model, we calculate several metrics such as Accuracy, Precision, Recall, and F1 Score. These metrics provide us with a comprehensive understanding of our model from different aspects, enabling us to make informed decisions and implement potential improvements or adjustments to our model.

2.5. Experiment 1:Standard Perceptron Model

We ran the model using the standard perceptron model mentioned above. The result obtained from this model serves as an understanding of the baseline performance of the Perceptron when applied to the Pima Indians Diabetes dataset.

2.5.1 Result and Discussion:

Metric	Accuracy	Precision	Recall	F1 Score
Value	0.67	0.58	0.68	0.58

Table 3. Results of Standard Perceptron Model

The result obtained is not good enough for the medical classification task, this happened because the model is not complex enough, and the step function in the perceptron uses strict binary prediction, which can lead to inconsistencies. Improvements in this area may yield better results.

2.5.2 Limitation of Standard Perceptron Model

The error in the standard perceptron model is calculated as the difference between the actual value and the predicted value. It is given as:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta(y_{\text{act}} - y_{\text{pred}})\mathbf{x}$$

The following table shows the error values of all possible actual and predicted target values.

$(y_{\textbf{actual}}, y_{\textbf{pred}})$	(1, 1)	(0, 0)	(1, 0)	(0, 1)
Error	0	0	1	-1

Table 4. Errors in Perceptron Model

As we can see the error has discrete values, implying the perceptron's updates are step-based meaning that the weight changes are discrete. This can lead to poor convergence and instability, especially in the case of noisy data.

2.6. Experiment 2: Introducing Binary Cross Entropy Loss and Sigmoid Function

To overcome the problem of discrete weight changes, we use a sigmoid activation function with binary cross-entropy loss to smooth the weight updates.

2.6.1 Sigmoid Activation Function

Mathematically the sigmoid can be written as:

$$y_{\text{pred}} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

This function squashes the result in between a range of 0 to 1. This, in effect, makes the model output a probability between 0 to 1, allowing smoother weight updates rather than discrete jumps [4].

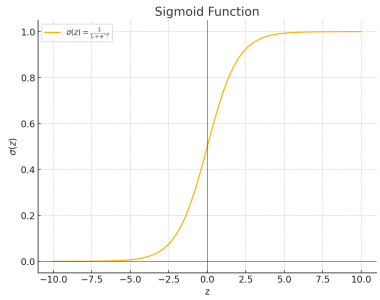


Figure 3. Sigmoid Activation Function

2.6.2 Binary Cross Entropy Loss

The error in this step is measured using the binary cross entropy loss function, this tells us how well the model's probability prediction matches actual labels. This is defined as:

$$\begin{aligned}\text{BCE}(y_{\text{actual}}, y_{\text{pred}}) = &- [y_{\text{actual}} \log(y_{\text{pred}})] \\ &- [(1 - y_{\text{actual}}) \log(1 - y_{\text{pred}})]\end{aligned}$$

To carry out the update of weights, we need the gradient of the loss function. The gradient follows from taking the partial derivative of the binary cross-entropy loss with respect to the weights. After applying the chain rule, this leads to the expression:

$$y - \hat{y}$$

where y is the actual label and $\hat{y}$ is the predicted probability, coming out from the sigmoid function. This essentially calculates the difference between the actual value and the predicted probability, which becomes the error term to correct the weights.

The update rule then becomes:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta(y - \hat{y})\mathbf{x}$$

The key difference here is that, $\hat{y}$ is the output of the sigmoid function, which outputs a probability value between 0 and 1. This means the update would be smoother, and it is based on how confident the model is about the prediction.

2.6.3 Result and Discussion:

Metric	Accuracy	Precision	Recall	F1 Score
Value	0.77	0.73	0.56	0.63

Table 5. Results of Perceptron with Sigmoid and BCE Loss

The result obtained here shows that the model performs quite better overall, especially in terms of accuracy and precision. This shows that the model indeed improved by using Sigmoid activation and binary cross entropy loss. The result obtained here tells us that the model is very reliable in predicting when someone has diabetes. However, low recall, means that it missed some true cases.

2.6.4 Limitations

The model gets more samples of individuals who do not have diabetes, hence it may become more biased towards predicting no diabetes often. This is the reason that leads to low recall values because the model struggles in classifying recognizing the few positive cases. To address this issue of class imbalance, we may introduce the concept of assigning class weights while updating the weights.

2.7. Experiment 3: Improving the model further(Handling Imbalance Class) by introducing class weights

In a dataset with an imbalanced class, for example in our case we have more people with no diabetes compared to people who have diabetes, the model predicts majority class i.e. no diabetes very often. To balance such scenarios, we will introduce class weights. By default, the class weights are calculated as:

$$\text{Class Weight for Class i} = \frac{\text{Number of Samples}}{\text{Number of Samples in Class i}}$$

For example:

- Weight for Class 0 (no diabetes): $\frac{768}{500} = 1.536$
- Weight for Class 1 (diabetes): $\frac{768}{268} = 2.865$

This means class 1 (diabetes) is weighted more heavily (2.865), so errors in predicting diabetes are given more importance than errors in predicting no diabetes.

2.7.1 Result and Discussion:

By trying different values, we set class weights to be 1 for the '0' class and 1.5 for the '1' class, here we are telling the model to give more importance to class 1 diabetic cases during the training.

Metric	Accuracy	Precision	Recall	F1 Score
Value	0.76	0.66	0.66	0.66

Table 6. Results of Perceptron with Class Weights

It shows significant improvement in the balancing model's prediction, particularly in Recall and F1 scores. The recall and precision both reached 66%, which means that the model now is better at identifying diabetes cases while still making accurate predictions overall.

2.7.2 Limitation

While we have made several improvements, there are still some limitations that affect the performance of the model. These limitations are:

- It is unable to learn nonlinear relationships between the features. A simple linear boundary may not be able to capture all the patterns.
- While we have addressed the issue of class imbalance by using class weights, it remains a challenge. The model may still miss some patterns in the minority class (e.g., diabetes) or may not fully generalize to new or unseen data.

2.8. Experiment 4: Testing Advanced Models

In this experiment, we implemented several other machine learning models, namely XGBoost, Random Forest, Logistic Regression, and Support Vector Classifier, have been implemented to test performance on the Pima Indian Diabetes dataset. Several works have compared the performances of XGBoost, Random Forest, Logistic Regression, and SVC. For instance, Fernandez-Delgado et al. performed an extended comparison between these models, outlining different strengths for the different classification tasks [8].

Model	Accuracy	Precision	Recall	F1 Score
XGBoost	0.74	0.63	0.63	0.63
Random Forest	0.76	0.70	0.56	0.62
Logistic Regression	0.76	0.63	0.72	0.67
SVC	0.74	0.61	0.73	0.66

Table 7. Comparison of Advanced Models

These models, serve as useful benchmarks, to compare our perceptron model against them. When comparing these metrics with our perceptron model with class weights,our model performed better than advanced models like Logistic Regression, Random Forest, and XGBoost, and offered stronger performance in some areas.

3. Link of Code and Dataset

https://github.com/chaudharysk/Perceptron.

4. Conclusion

We have gone through a series of experiments, exploring the ways of improving the standard perceptron model on the Diabetes dataset. In experiment 1, using the standard perceptron model, we achieved a very moderate performance, which highlighted the need to handle complex relationships and balance the imbalanced class. In experiment 2, we introduced the sigmoid activation function and binary cross-entropy loss, the model achieved moderate performance in precision and accuracy but the recall still remained low. In experiment 3, we address the issue of class imbalance by introducing the class weights concept, it significantly improved the model's recall and F1 score, balancing its ability to predict diabetes cases more precisely. By this improvement, we showed that class weights allowed the model to handle the imbalance between diabetes and non-diabetes efficiently. Finally, in experiment 4, we implemented several advanced models like XGBoost, Random Forest, Logistic Regression, and Support Vector Machine Classifier to compare these with our improvised perceptron model.

While the perceptron with class weights performed pretty well in the overall case, some models such as Logistic Regression and Random Forest, performed slightly better in specific areas, such as recall and precision, respectively. Overall, the advanced model can capture complex relationships slightly better, but with proper adjustment, even simple models can be competitive and effective.

References

[1] A. Susana, T. Arias, and H. M. Kattyta, "Diabetes prediction using the multilayer perceptron algorithm: An analysis of the diabetes dataset," *Medwave*, vol. 23, no. S1, pp. eUTA060–eUTA060, 2023. 1

[2] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain," *Psychological Review*, vol. 65, pp. 386–408, 1958. 1

[3] UCI Machine Learning Repository, "Pima indians diabetes database," Available at: https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database, 1988, accessed: Oct. 1, 2024. 1, 3

[4] S. Haykin, *Neural Networks: A Comprehensive Foundation*, 1st ed. USA: Prentice Hall PTR, 1994. 2, 3

[5] A. Novikoff, "On convergence proofs on perceptrons," vol. 12, pp. 615–622, 1962. 3

[6] "LIBSVM: A library for support vector machines," https://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/binary.html, 2011, accessed: Oct. 2, 2024. 3

[7] O. Troyanskaya, M. Cantor, G. Sherlock, T. Hastie, R. Tibshirani, D. Botstein, and R. Altman, "Missing value estimation methods for dna microarrays," *Bioinformatics*, vol. 17, no. 6, pp. 520–525, Jul. 2001. 3

[8] M. Fernández-Delgado, E. Cernadas, S. Barro, and D. Amorim, "Do we need hundreds of classifiers to solve real world classification problems?" *Journal of Machine Learning Research*, vol. 15, no. 90, pp. 3133–3181, 2014. 5